

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

Scenario 1's Question: Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

Answer:

```
SET SERVEROUTPUT ON;

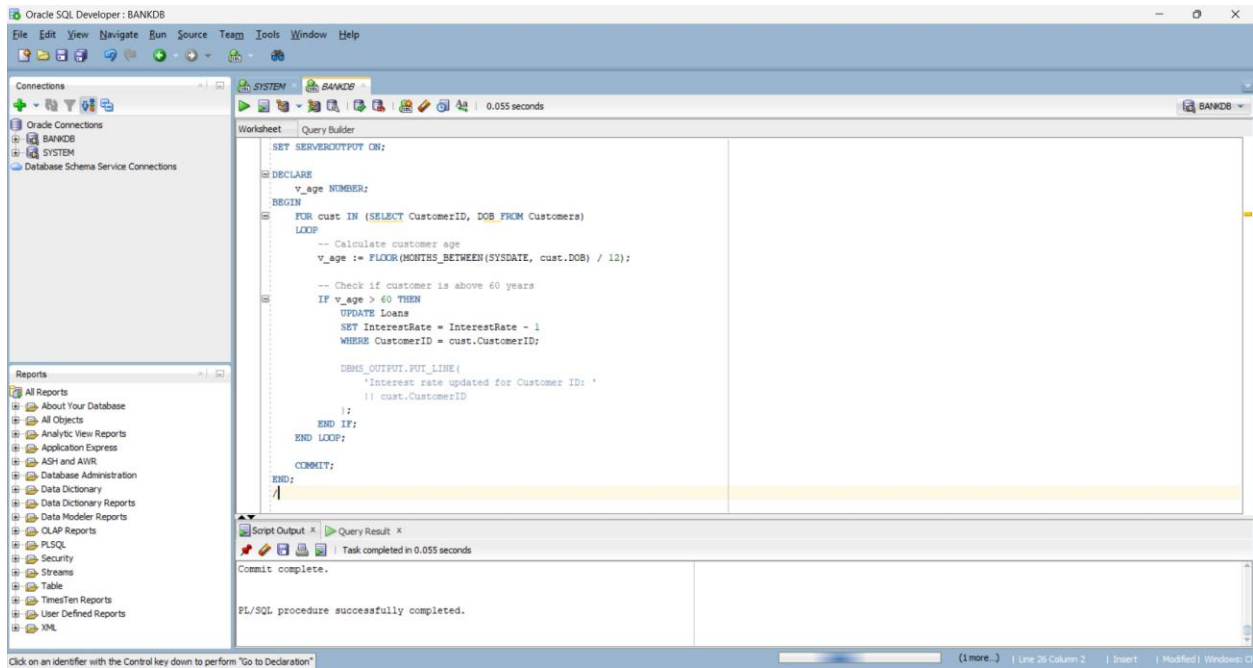
DECLARE
    v_age NUMBER;
BEGIN
    FOR cust IN (SELECT CustomerID, DOB FROM Customers)
    LOOP
        -- Calculate customer age
        v_age := FLOOR(MONTHS_BETWEEN(SYSDATE, cust.DOB) / 12);

        -- Check if customer is above 60 years
        IF v_age > 60 THEN
            UPDATE Loans
            SET InterestRate = InterestRate - 1
            WHERE CustomerID = cust.CustomerID;

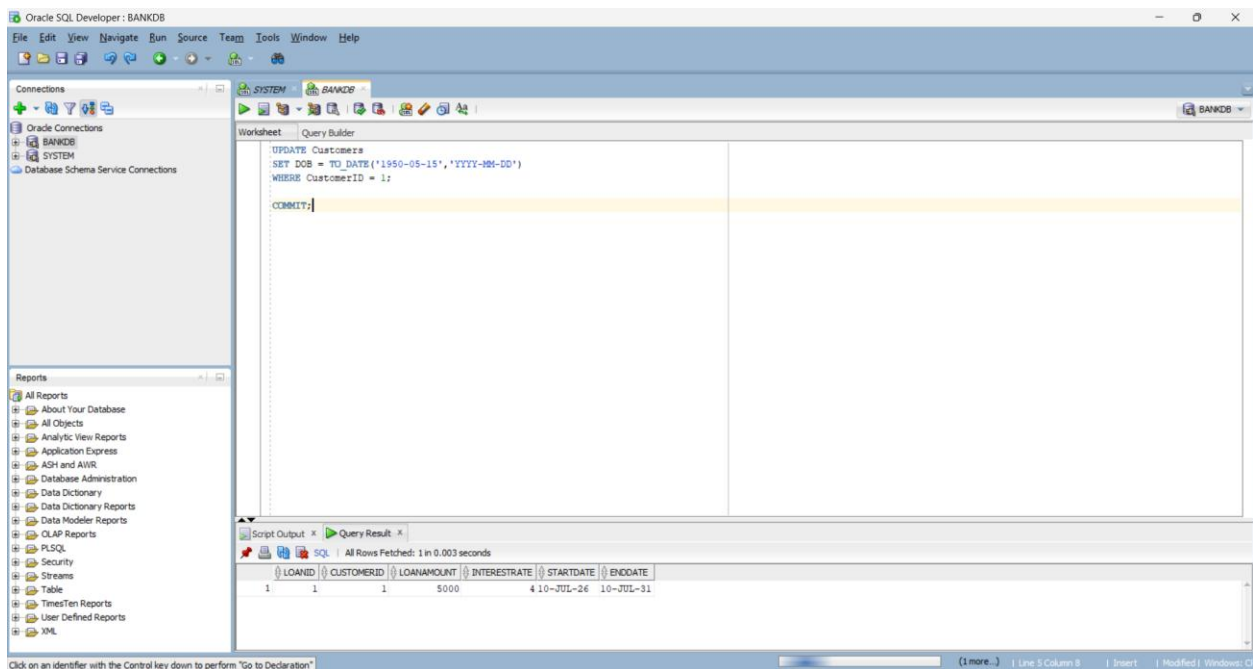
            DBMS_OUTPUT.PUT_LINE(
                'Interest rate updated for Customer ID: '
                || cust.CustomerID
            );
        END IF;
    END LOOP;

    COMMIT;
END;
/
```

Execution of Programming Language Script:



Verification Query



Scenario 2: A customer can be promoted to VIP status based on their balance.

Scenario 2's Question: Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

Answer:

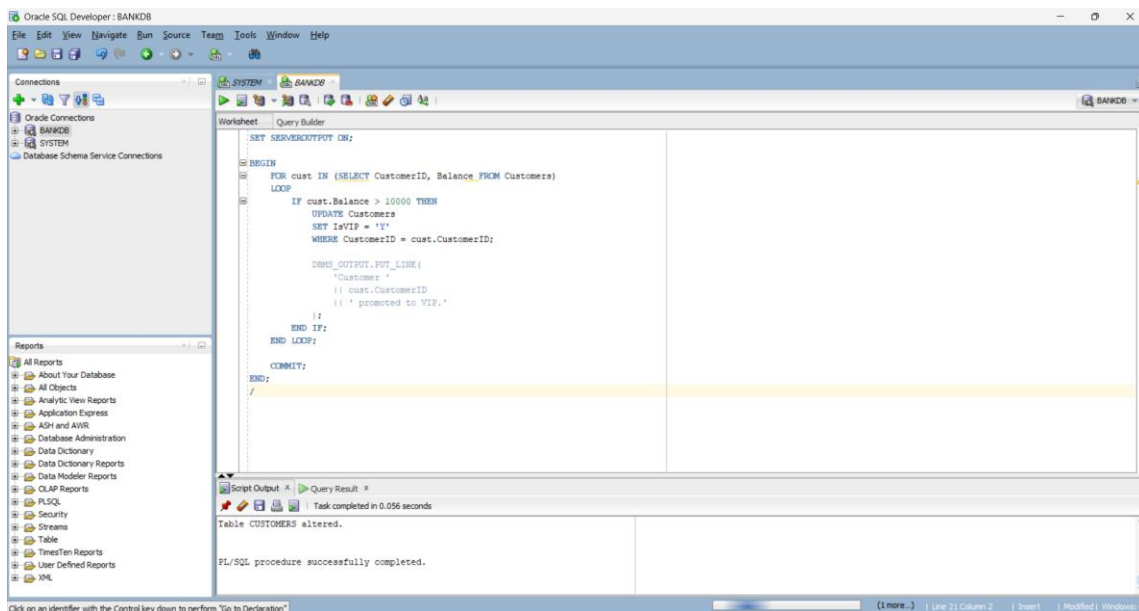
```
SET SERVEROUTPUT ON;

BEGIN
  FOR cust IN (SELECT CustomerID, Balance FROM Customers)
  LOOP
    IF cust.Balance > 10000 THEN
      UPDATE Customers
      SET IsVIP = 'Y'
      WHERE CustomerID = cust.CustomerID;

      DBMS_OUTPUT.PUT_LINE(
        'Customer '
        || cust.CustomerID
        || ' promoted to VIP.'
      );
    END IF;
  END LOOP;

  COMMIT;
END;
/
```

Execution of Programming Language Script:



Verification Query

The screenshot displays the Oracle SQL Developer interface for a database named BANKDB. The main window is the Worksheet, which contains a SQL query in the Query Builder:

```
SELECT CustomerID,  
       Name,  
       Balance,  
       IsVIP  
FROM Customers;
```

Below the query, the Query Results pane shows the output of the query. It indicates that all rows were fetched in 0.004 seconds. The results are displayed in a table with four columns: CUSTOMERID, NAME, BALANCE, and ISVIP.

CUSTOMERID	NAME	BALANCE	ISVIP
1	John Doe	1000 (null)	
2	Jane Smith	1500 (null)	

The interface also includes a Connections pane on the left showing the current connection to BANKDB, and a Reports pane at the bottom left listing various report types.

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

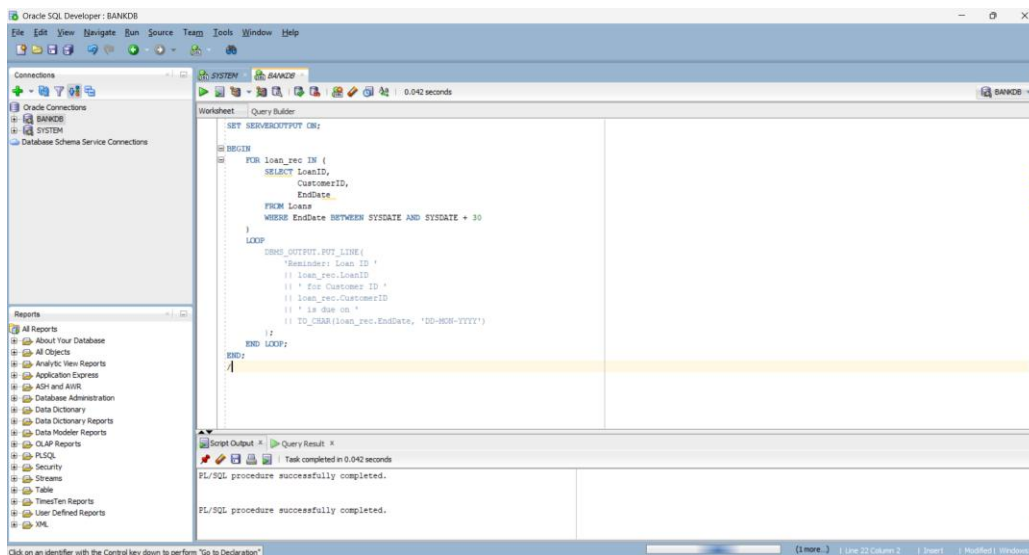
Scenario 3's Question: Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

Answer:

```
SET SERVEROUTPUT ON;

BEGIN
  FOR loan_rec IN (
    SELECT LoanID,
           CustomerID,
           EndDate
    FROM Loans
    WHERE EndDate BETWEEN SYSDATE AND SYSDATE + 30
  )
  LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Reminder: Loan ID '
      || loan_rec.LoanID
      || ' for Customer ID '
      || loan_rec.CustomerID
      || ' is due on '
      || TO_CHAR(loan_rec.EndDate, 'DD-MON-YYYY')
    );
  END LOOP;
END;
```

Execution of Programming Language Script



Verification Query:

The screenshot displays the Oracle SQL Developer interface with the 'BANKDB' connection selected. The main workspace shows a PL/SQL procedure named `DBMS_OUTPUT.PUT_LINE` within a loop, designed to print loan reminders. The procedure has been executed successfully, as indicated by the 'Script Output' and 'Query Result' tabs.

Script Output:

```
Reminder: Loan ID 1 for Customer ID 1 is due on 25-JUL-2026
```

Query Result:

```
PL/SQL procedure successfully completed.
```

Dbms Output:

```
BANKDB  
Reminder: Loan ID 1 for Customer ID 1 is due on 25-JUL-2026
```

The status bar at the bottom indicates the current position is at Line 22, Column 2, and provides a hint: 'Click on an identifier with the Control key down to perform "Go to Declaration"'.